25COP922 Coursework
Part Time Data Science MSc
Student ID- F416013

Executive Summary

This project presents a comprehensive design and implementation of a Railway Reservation System, structured through a series of deliverables that demonstrate a robust application of database system principles. This project begins with Deliverable 1, which involves the creation of the Entity Relationship Diagram (ERD) that models the key entities and relationships within the system, such as **PASSENGER**, **BOOKING**, and **TRAIN SCHEDULE**, providing a foundational blueprint for the database schema.

In Deliverable 2, the ERD is translated into a detailed schema, capturing strong entities such as **TRAIN** and **STATION** and weak entities such as **SEAT** and **TRAIN SCHEDULE**. This schema is meticulously designed with constraints, primary and foreign keys to ensure data integrity and efficient query performance. Furthermore, there are details regarding triggers and indexing strategies, these are also essential parts of the schema.

In Deliverable 3, this focuses on identifying functional dependencies and ensuring the schema adheres to Boyce-Codd Normal Form (BCNF). Each entity is evaluated for compliance, with key findings indicating that all entities satisfy BCNF, this therefore eliminating redundancy and anomalies.

The performance and representational power of the schema are analysed in Deliverable 4, highlighting its ability to model complex operational workflows while maintaining semantic clarity and supporting high-volume queries. The use of strong and weak entities, along with targeting indexing, enhances both te schemas expressiveness and performance.

Deliverable 5 showcases the practical application of the schema through SQL queries that address various operational and analytical tasks, such as determining seat booking and train revenue. These queries demonstrate the schema's robustness and its capacity to support diverse functionalities.

Table of Contents

Glossary

BCNF	**Boyce-Codd Normal Form** is a schema design principle within database normalisation aimed at eliminating "anomalous" update behaviours by ensuring that "independent relationships" are placed into "independent relations". A relation schema is in BCNF if, for all non-trivial functional dependencies is a super key (Bernstein and Goodman, 1980).
ERD	An **Entity Relationship Diagram** is a graphical representation used to model the visible semantics of a database, illustrating the relationships between their entities and their attributes (Jajodia and Springsteel, 1983).
FD	A **Functional Dependency** is a formal expression within the relational model that implies that for any two tuples in a relation, if the tuples agree with the attributes in X they must also agree on the attributes in Y (Vianu, 1987).
FK	A **Foreign Key** is an inclusion dependency that establishes a referential integrity constraint between two tables by referencing a primary key in another table. It ensures that the values in the foreign key column(s) match values references in the primary key column. A foreign key must be inclusion dependant, meaning each value in the left- hand side of a foreign key must be included in the value set of its right-hand side (Jiang and Naumann, 2019).
PK	A **Primary Key** is a unique column combination within a relational database schema that ensures entity integrity by containing only unique, non-null value entries for each tuple in a relation. It is a minimal set of attributes that uniquely identifies each record in a table. Each table should only have one primary key (Jiang and Naumann, 2019).

Introduction

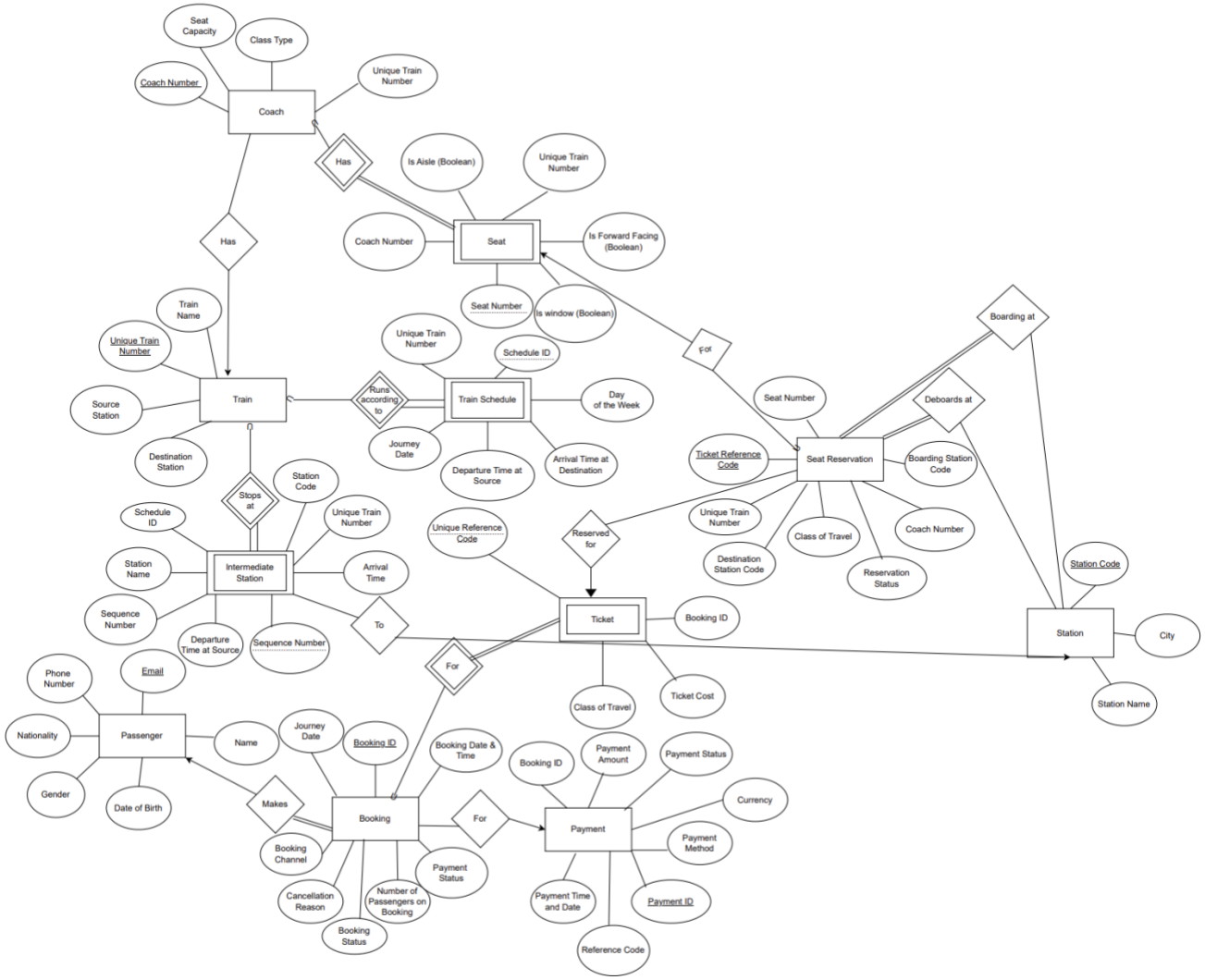
This project consists of a detailed exploration of a railway reservation system, focusing on the design and implementation of a robust database schema. Throughout this document, all entities are consistently represented in capital letters and bold, such as **PASSENGER**, and **BOOKING**, to clearly distinguish them from attributes, which are denoted in bold and lower case such as **Email**, and **Unique Train Number**. This formatting choice enhances readability and ensures clarity in the representation of the database structure.

This project is structured into key deliverables, each addressing a specific aspect of the database system. Initially, an Entity Relationship Diagram (ERD) has been developed to model the relationships and attributes of the system, providing a visual foundation for the subsequent schema. Following this, the ERD is translated into a detailed schema, capturing both strong and weak entities, and incorporating constraints, primary and foreign keys, and indexing strategies to ensure data integrity and performance.

The next phase involves identifying functional dependencies and ensuring the schema adheres to Boyce-Codd Normal Form (BCNF), thereby eliminating redundancy and anomalies. This is followed by an analysis of the schema's performance and representational power, highlighting its ability to model complex operational workflows while maintaining semantic clarity.

Finally, practical application of the schema us demonstrated through a series of SQL queries, showcasing its capability to handle diverse analytical and operational tasks. These components collectively illustrate a discipline application of relational database principles, aimed at achieving a scalable and maintainable railway system.

Deliverable 1- Entity Relationship Diagram



Assumptions:

- The boolean attributes **Is Aisle**, **Is Window**, and **Is Forward Facing** are sufficient to capture all seat preferences for passengers.
- One-to-one relationship between **TICKET** and **SEAT RESERVATION** meaning for every seat reserved there is a unique **TICKET** associated with it.

Deliverable 2- A schema resulting from the ERD

TRAIN (STRONG ENTITY)

Unique Train Number (PK)
Source Station Code (FK) to **STATION** (**Station Code**)
Destination Station Code (FK) to **STATION** (**Station Code**)
Train Name

Constraints

- Ensure that the source and destination stations are not the same.

Primary Key

- **Unique Train Number** uniquely identifies each train in the system so therefore is the primary key.

Foreign Key

- **Source Station Code** and **Destination Station Code** reference valid stations from the **STATION** entity.

Indexes

- An index on **Source Station Code** to speed up queries that search for trains departing from a particular station.
- An index on **Destination Station Code** to improve performance when finding trains arriving at a specific station.

Triggers

- Before inserting a **TRAIN**, verify that both **Source Station Code** and **Destination Station Code** exist in the **STATION** table. If not, reject the insert.
- Before inserting or updating a **TRAIN**, check that the **Source Station Code** is not equal to the **Destination Station Code**. If they are the same, block the operation and return an error.
- After inserting **TRAIN**, automatically create seven **TRAIN SCHEDULE** rows, one for each day of the week, with empty departure and arrival times ready to be filled in later.

STATION (STRONG ENTITY)

Station Code (PK)
Station Name
City

Constraints

- **Station Code** must not be empty and must be a 3-letter code e.g EDB.
- **Station Name** must not be empty.
- **City** field must not be empty.

Primary Key

- **Station Code** uniquely identifies each station and is referenced by other entities to define routes and station stop locations.

Foreign Key

- **STATION** has no foreign keys as it is a strong foundational entity that does not depend on any other table for its definition or identity.

Indexes

- An index on **Station Code** is automatically created as this is the primary key.
- Could have an index on **City** to improve performance when filtering stations by location.

Triggers

- Before deleting a **STATION**, check whether it is referenced in **TRAIN**, **INTERMEDIATE STATION**, or **SEAT RESERVATION.** If it is then block the deletion to preserve referential integrity.
- Before inserting or updating a **STATION**, automatically convert **Station Code** to uppercase to ensure consistency.
- Before inserting a **STATION**, check whether a **STATION** with the same **Station Name** exists in the same **City**. If so, reject the insert to avoid confusion.

COACH (STRONG ENTITY)

Coach Number (PK)
Unique Train Number (FK) to **TRAIN** (**Unique Train Number**)
Class Type
Seat Capacity

Constraints

- **Class Type** is restricted to valid travel classes, such as “First”, “Second”, “Sleeper” and “Standard” to maintain consistency.

Primary Key

- **Coach Number** is the primary key for the **COACH** entity, as it uniquely identifies each coach on a **TRAIN**.

Foreign Key

- **Unique Train Number** acts as the foreign key within the **COACH** entity as it references the **Unique Train Number** attribute in the **TRAIN** entity.

Indexes

- An index on **Unique Train Number** would improves performance when retrieving all coaches for a given train.
- An index on **Class Type** may help if filtering by travel class is common.

Triggers

- Before inserting a **COACH**, check whether a coach with the same **Coach Number** already exists for that **Unique Train Number**. If so, block the insert to avoid duplication.
- After inserting a **COACH**, if **Class Type** is missing, automatically set it to “Standard”.
- Before deleting a **COACH**, check whether any **SEAT** rows exist for that **Unique Train Number** and **Coach Number**. If so, block the deletion to preserve the seat data.

SEAT (WEAK ENTITY)

Unique Train Number (FK) to **TRAIN** (**Unique Train Number**)
Coach Number (FK) to **COACH** (Coach Number)
Seat Number (Discriminator)
Is Window (Boolean)
Is Forward Facing (Boolean)
Is Aisle (Boolean)

Constraints

- Seat attributes such as **Is Window, Is Forward Facing**, and **Is Aisle** must be valid Boolean values (True or False).
- **Is Forward Facing** is not a static entity. This is dependent on the direction of the journey.
- **Seat Number** must be a positive integer to prevent invalid entries.

Primary Key

- Due to **SEAT** being a weak entity, the primary key comes from **COACH** and **TRAIN**, these being **Unique Train Number**, **Coach Number** and **Seat Number**.

Foreign Key

- **Unique Train Number** acts as a foreign key as it references the **Unique Train Number** attribute in the **TRAIN** entity.
- **Coach Number** is also a foreign key, linking each seat to its corresponding coach in the **COACH** entity.

Discriminator

- **SEAT** is a weak entity that depends on **COACH** and **TRAIN**. Its primary key is a composite of **Unique Train Number** and **Coach Number**, where **Seat Number** acts as the discriminator, therefore uniquely identifying each seat within a coach.

Indexes

- Index on **Unique Train Number** and **Coach Number** would speed up queries that list all seats in a coach.
- Index on **Unique Train Number**, **Coach Number** and **Seat Number** would be automatic as these are the primary keys.
- Index on **Is Window**, **Is Aisle**, **Is Forward Facing** could be used to frequently filter by seat type.

Triggers

- Before inserting a **SEAT**, check whether a seat with the same **Seat Number** already exists for that **Unique Train Number** and **Coach Number**. If so, block the insert.
- Before deleting a **SEAT**, check whether any **SEAT RESERVATION** rows reference it. If they do, then block the deletion to preserve reservation integrity.
- After inserting a **SEAT**, automatically set **Is Window** or **Is Aisle** or **Is Forward Facing** based on a predefined layout pattern.

TRAIN SCHEDULE (WEAK ENTITY)

Unique Train Number (FK) to **TRAIN** (**Unique Train Number**)
Schedule ID (Discriminator)
Day of Week
Departure Time at Source
Arrival Time at Destination
Journey Date

Constraints

- Ensure that **Day of the Week** is a valid day and that **Departure Time at Source** and **Arrival Time at Destination** are correctly formatted.
- **Departure Time at Source** should occur before **Arrival Time at Destination**.

Primary Key

- **Unique Train Number** forms the primary key in **Train Schedule** as it is inherited from the strong entity **TRAIN**.

Foreign Key

- **Unique Train Number** acts as the foreign key in **TRAIN SCHEDULE** as it references the **Unique Train Number** attribute in the **TRAIN** entity.

Discriminator

- **TRAIN SCHEDULE** is a weak entity that depends on **TRAIN**. Its primary key is a composite of **Unique Train Number** and **Day of the Week**, where **Schedule ID** is the discriminator and uniquely identifies each schedule entry for a train.

Indexes

- An index on **Unique Train Number** would improve performance when listing all schedules for a train.
- The composite primary key also acts as an index for fast lookups by **Unique Train Number** and **Day of the Week**.

Triggers

- Before inserting a **TRAIN SCHEDULE**, check whether a row already exists for the same **Unique Train Number** and **Day of the Week**. If it does then block the insert, to avoid duplication.
- After inserting a **TRAIN SCHEDULE**, if **Departure Time at Source** and **Arrival Time at Destination** are null, then automatically set them to default placeholders, such as 00:00 to ensure completeness.
- Before deleting a **TRAIN**, check whether any **TRAIN SCHEDULE** rows exist for that **Unique Train Number**. If they do, then block the deletion to preserve any schedule data.

INTERMEDIATE STATION (WEAK ENTITY)

Unique Train Number (FK) to **TRAIN** (**Unique Train Number**)
Sequence Number (Discriminator)
Station Code (FK) to **STATION** (**Station Code**)
Arrival Time
Departure Time at Source

Constraints

- Ensure that **Sequence Number** is greater than zero and that **Arrival Time** and **Departure Time at Source** must be logically consistent and correctly formatted.

Primary Key

- **Unique Train Number** is the primary key coming from the strong entity **TRAIN**.

Foreign Key

- **Unique Train Number** acts as the foreign key in **INTERMEDIATE STATION** as it references the **Unique Train Number** attribute in the **TRAIN** entity.
- **Station code** also acts as the foreign key in **INTERMEDIATE STATION** as it references the **Station code** attribute in the **STATION** entity.

Discriminator

- **INTERMEDIATE STATION** is a weak entity that depends on **TRAIN**. Its primary key is a composite of **Unique Train Number** and **Sequence Number**, where **Sequence Number** is the discriminator and uniquely identifies each intermediate stop for a train.

Indexes

- An index on **Unique Train Number** would improve performance when retrieving all intermediate stops for a train. The composite primary key also acts as in index for fast lookups by order of train and stops.

Triggers

- Before inserting an **INTERMEDIATE STATION**, check whether a row already exists with the same **Unique Train Number** and **Sequence Number**. If so, block the insert to avoid duplication.
- Before deleting **STATION**, check whether it is reference in **INTERMEDIATE STATION**. If it is, then block the deletion to preserve route integrity.
- After inserting an **INTERMEDIATE STATION,** if **Sequence Number** is null, then automatically assign the next available number for that **Unique Train Number**.

PASSENGER (STRONG ENTITY)

Email (PK)
Name
Date of Birth
Phone Number
Nationality
Gender

Constraints

- Ensure that **Email** and **Name** entities are not empty, and that **Date of Birth** is no earlier than today to prevent invalid entries.

Primary Key

- **Email** is the primary key as this uniquely identifies each user and allows for consistent referencing across bookings and tickets.

Foreign Key

- Due to **PASSENGER** being a strong entity and not referencing any other table it does not have any foreign keys. Although it is related to **BOOKING**, this relationship does not require **PASSENGER** to store any foreign keys as **BOOKING** is also a strong entity with its own attributes.

Indexes

- An index on **Email** is automatically created as it's the primary key.
- An index on **Date of Birth** may help if the system uses age-based filtering or eligibility checks.

Triggers

- Before inserting a **PASSENGER**, check that the **Email** field matches a valid email pattern. If it does not, then reject the insert.
- Before inserting a **PASSENGER**, check whether a row already exists with the same **Email**. If it does, then block the insert to avoid duplication.
- After inserting a **PASSENGER**, calculate their age using **Date of Birth** and automatically flag if they are under 18, this can then be used for eligibility of discounts or parental consent requirement.

BOOKING (STRONG ENTITY)

Booking ID (PK)
Journey Date
Booking Date and Time
Number of Passengers
Payment Status
Booking Channel
Cancellation Reason
Booking Status

Constraints

- Ensure that **Booking Date and Time** are not in the future, **Journey Date** is not in the past, and **Payment Status** is restricted to valid options to prevent invalid entries.

Primary Key

- **Booking ID** is the primary key for **BOOKING** as this uniquely identifies each transaction and supports connections to tickets, payments and seat reservations.

Foreign Key

- **BOOKING** is a strong entity and does not contain any foreign keys because none of its attributes reference another table.

Indexes

- **Booking ID** is the primary key and therefore the primary index for the **BOOKING** entity.
- An additional index such as **Journey Date**, **Booking Date and Time**, **Payment Status**, and **Booking Channel** may help improve the efficiency of searching and filtering **BOOKING** records.
-

Triggers

- Before inserting a **BOOKING**, check that **Journey Date** is today or later. If not, then reject the insert.
- After inserting a **BOOKING**, if **Booking Status** is null, automatically set it to “pending”.
- Before inserting a **BOOKING**, check whether a booking already exists for the same **Passenger Email** (from the **PASSENGER** attribute) and **Journey Date**. If it does then block the insert, to avoid duplication.

TICKET (WEAK ENTITY)

Booking ID (FK) to **BOOKING** (**Booking ID**)
Unique Reference Code (Discriminator)
Class of Travel
Ticket Cost
Unique Train Number (FK) to **SEAT** (**Unique Train Number**)
Seat Number (FK) to **SEAT** (**Seat Number**)

Constraints

- Each **TICKET** must be linked to a valid booking through **Booking ID**.
- The **Unique Reference Code** must be unique within each booking.
- **Unique Train Number**, **Coach Number** and **Seat Number** must correspond to a valid seat in the **SEAT** entity.
- Whilst **Ticket Cost** can technically be derived from distance and class it is stored as a static value at the moment of purchase, therefore meaning if the price changes the price they paid at the time of booking will be still stored.

Primary Key

- Due to **TICKET** being a weak entity, the primary key is a composite of **Booking ID** and **Unique Reference Code**.

Foreign Key

- **Booking ID** acts as the foreign key in **TICKET** as it references the **Booking ID** attribute in the **BOOKING** entity.
- **Unique Train Number**, **Coach Number**, and **Seat Number** act as foreign keys as they reference their corresponding attributes in the **SEAT** entity, ensuring each ticket is linked to a valid **SEAT**.

Discriminator

- The **Unique Reference Code** acts as the discriminator for the **TICKET** weak entity, uniquely identifying each ticket within a **BOOKING**.

Indexes

- An index on **Booking ID** would improve performance when retrieving all tickets for a booking.
- An index on **Seat Number** would support efficient validation of seat availability and joins with **SEAT RESERVATION**.

Triggers

- Before inserting a **TICKET**, check whether a ticket already exists for the same **Booking ID**, and **Unique Reference Code**.
- Before inserting a **TICKET**, verify that the seat us not already reserved for the same journey.
- After inserting a **TICKET**, automatically set its status based on the **BOOKING Payment Status** or **Booking Status**.

SEAT RESERVATION (STRONG ENTITY)

Ticket Reference Code (FK & PK) to **TICKET** (**Unique Reference Code**)
Class of Travel
Unique Train Number (FK) to **SEAT** (**Train Number**)
Coach Number (FK) to **SEAT** (**Coach Number**)
Seat Number (FK) to **SEAT** (**Seat Number**)
Boarding Station Code (FK) to **STATION** (**Station Code**)
Destination Station Code (FK) to **STATION** (**Station Code**)
Reservation Status

Constraints

- The **Boarding Station Code** and **Destination Station Code** must be different to ensure a valid journey.
- **Ticket Reference Code** must be unique, ensuring each reservation is uniquely identifiable and linked to exactly one ticket.
- **Unique Train Number**, **Coach Number**, and **Seat Number** must correspond to a valid seat in the **SEAT RESERVATION** entity.
-

Primary Key

- **Ticket Reference Code** is the primary key as it uniquely identifies each **SEAT RESERVATION.** It is also functions as a foreign key to the **TICKET** entity. This reflects the one-to-one relationship between **TICKET** and **SEAT RESERVATION**, where each reservation corresponds to exactly one ticket.

Foreign Key

- **SEAT RESERVATION** references the **TICKET**, **SEAT**, and **STATION** entities to associate each reservation with a specific ticket, a valid seat, and the journey's start and end stations.
- **Unique Train Number**, **Coach Number**, and **Seat Number** reference the SEAT entity, while **Boarding Station Code** and **Destination Station Code** reference the **STATION** entity.

Indexes

- An index on **Ticket Reference Code** would already be made as this is the primary key. This will improve performance when retrieving the reservation associated with a specific ticket.
- Indexes on **Unique Train Number**, **Coach Number**, and **Seat Number** improve efficiency when checking seat availability or joining with the **SEAT** entity.
- Indexes on **Boarding Station Code** and **Destination Station Code** would support faster searching for station-based queries.

Triggers

- Before inserting a **SEAT RESERVATION**, check whether the seat is already reserved for the same **Unique Train Number** and **Journey Date**. If it is then, block the insert.
- After inserting a **SEAT RESERVATION**, verify that no existing reservation uses the same **Ticket Reference Code**, if one exists then block the insert to avoid duplication.
- After inserting a **SEAT RESERVATION**, if **Reservation Status** is null, automatically set it to “pending”, this will maintain consistent default behaviours.

PAYMENT (STRONG ENTITY)

Payment ID (PK)
Booking ID (FK) to **BOOKING** (**Booking ID**)
Payment Amount
Payment Method
Payment Date and Time
Payment Status
Currency
Reference Code

Constraints

- **Payment Amount** must be greater than zero.
- **Payment Status** must contain a valid value such as Pending, Completed, Refunded, or Failed.
- **Payment Method** must be one of the accepted options, such as Debit Card, Credit Card, Gift Card.
- **Booking ID** must reference an existing booking to ensure every payment is linked to a valid journey.

Primary Key

- **Payment ID** is the primary key as this uniquely identifies each transaction.

Foreign Key

- **Booking ID** acts as the foreign key linking each payment to the **BOOKING** entity, ensuring that each transaction is associated with a specific passenger booking.

Indexes

- An index on **Booking ID** improves performance when retrieving payments for a specific booking.
- An index on **Payment Status** would help when listing pending or refunded transactions.

Triggers

- Before inserting a **PAYMENT**, check whether a payment already exists for the same **Booking ID**. If it does, then block the insert to avoid duplication.
- After inserting a **PAYMENT**, if **Payment Status** is null, automatically set it to “Pending”.
- Before inserting a **PAYMENT**, check whether the associated booking has status “Cancelled”. If it does, then block the insert as this is no longer necessary.

Deliverable 3 – Identify all functional dependencies and use that information to decompose the schema (as needed) to make it BCNF

This deliverable identifies all functional dependencies (FDs) within the Railway Reservation System and evaluates each entity against Boyce-Codd Normal Form (BCNF). Functional dependency is a crucial concept in relational database design, whereby one attribute uniquely determines another. BCNF is a higher version of the Third Normal Form (3NF), addressing anomalies that are not managed by 3NF. This ensures that every determinant in a functional dependency is a candidate key, therefore preventing redundancy, anomalies and inconsistent data. By utilising the entities developed in deliverable 2, each table will be examined to identify its functional dependencies and determine if decomposition is necessary to achieve BCNF (Noor, 2021).

1. TRAIN

Functional Dependencies

Unique Train Number -> Source Station, **Destination Station**, **Train Name**

Explanation

Unique Train Number is the primary key and uniquely identifies each **TRAIN**. This therefore makes it a candidate key, ensuring the relation satisfies BCNF.

2. STATION

Functional Dependencies

Station Code -> Station Name, **City**

Explanation
Station Code is the primary key and uniquely identifies each **STATION**. This therefore makes it a candidate key, ensuring the relation satisfies BCNF.

3. COACH

Functional Dependencies

Unique Train Number, **Coach Number -> Class Type**, **Seat Capacity**

Explanation

The combination of **Unique Train Number** and **Coach Number** uniquely identifies each **COACH**. The candidate key would therefore be **Unique Train Number** and **Coach Number**. This relation therefore satisfies BCNF.

4. **SEAT** (Weak Entity)

Functional Dependencies

Unique Train Number, **Coach Number**, **Seat Number** -> **Is Window**, **Is Forward Facing**, **Is Aisle**

Unique Train Number, **Coach Number, Seat Number** -> **Class Type**, **Seat Capacity**

Explanation

For the first functional dependency for this weak entity, **SEAT**, the composite key, **Unique Train Number**, **Coach Number** and **Seat Number** uniquely identify each **SEAT** and determine the attributes. **Seat Number** acts as a discriminator to uniquely identify each **SEAT** within a **COACH**. For the second functional dependency, while this is not directly part of the **SEAT** entity, these attributes can be inferred through the relationship with the **COACH** entity. The composite key, **Unique Train Number** , **Coach Number** and **Seat Number**, act as the primary key for this weak entity, ensuring each **SEAT** is uniquely identifiable. The determinant, **Unique Train Number**, **Coach Number** and **Seat Number** is therefore a candidate key and satisfies BCNF.

5. **TRAIN SCHEDULE** (Weak Entity)

Functional Dependencies

Unique Train Number, **Schedule ID** -> **Journey Date**, **Departure Time at Source**, **Arrival Time at Destination**, **Day of the Week**

No attributes are determined solely by **Unique Train Number** or **Schedule ID** as it is a foreign key to **TRAIN**

Explanation

For the functional dependency for this weak entity, **TRAIN SCHEDULE**, the composite key **Unique Train Number** and **Schedule ID** uniquely identify each scheduled service and determine its attributes, **Journey Date**, **Departure Time at Source**, **Arrival Time at Destination** and **Day of the Week**. **Schedule ID** acts as the discriminator to uniquely identify each schedule within a given **TRAIN**. However, a second consideration, while this is not a functional dependency within **TRAIN SCHEDULE**, are train-level attributes, for example **Train Name** can be inferred by the relationship to the **TRAIN** entity and therefore should not be stored in **TRAIN SCHEDULE** to avoid redundancy. The composite key, **Unique Train Number** and **Schedule ID**, serves as the primary key for this weak entity, ensuring each schedule us uniquely identifiable. The determinant, **Unique Train Number** and **Schedule ID**, is therefore a candidate key, and since no clear subset of this key determines non-key attributes, and no non key attributes determines another, the relation satisfies BCNF.

6. **INTERMEDIATE STATION** (Weak Entity)

Functional Dependencies

Unique Train Number, **Schedule ID**, **Station Code**, **Sequence Number** -> **Arrival Time**, **Departure Time at Source**, **Day of the Week**, **Station Name**

Station Code -> **Station Name**

Explanation

The composite key, **Unique Train Number**, **Schedule ID**, **Station Code** and **Sequence Number**, uniquely identifies each stop and determines the stop-specific attributes, **Arrival Time**, **Departure Time at Source**, **Day of the Week** and **Station Name**. **Schedule ID** ties the stop to a particular service instance on the **TRAIN**. However, **INTERMEDIATE STATION** is not in BCNF unless **Station Name** is removed and derived via a join to **STATION**. This is due to it introducing a non-key determinant (**Station Code**) for a non-key attribute (**Station Name**). Therefore, **Station Name** should be removed from this entity and solely derived from the station entity to satisfy BCNF.

7. **PASSENGER**

Functional Dependencies

Email -> **Phone Number**, **Nationality**, **Gender**, **Date of Birth**, **Name**

Explanation

The **PASSENGER** entity is a strong entity with **Email** as the primary key. All the descriptive attributes are fully functionally dependant on **Email**, and no non-key attribute determines another. Due to the key being a single attribute there are no partial dependencies, no non-key determinants and no transitive dependencies. If the system later introduces a surrogate identifier, such as **Passenger ID** or enforces additional uniqueness constraints, such as a **Verified Phone Number**, these can function as alternate keys without affecting the BCNF status, provided all passenger attributes continue to be determined by a key.

8. **BOOKING**

Functional Dependencies

Booking ID -> **Booking Status**, **Journey Date**, **Booking Date and Time**, **Payment Status**, **Number of Passengers on a Booking**, **Cancellation Reason**, **Booking Channel**

Explanation

The **BOOKING** entity is a strong entity with **Booking ID** as its primary key. All descriptive attributes are fully functionally dependent on **Booking ID**. As the key is not composite, partial dependencies cannot arise, and no non-key attribute determines another non-key

attribute, for example **Booking Channel** does not determine **Journey Date**, every nontrivial functional dependency in **BOOKING** satisfies BCNF, without requiring decomposition.

9. **TICKET** (Weak Entity)

Functional Dependencies

Booking ID, Unique Reference Code -> Class of Travel, Ticket Cost

Booking ID -> no TICKET attributes alone as it is a foregin key to BOOKING

Unique Reference Code -> no determination alone as the code is only unique within a BOOKING

Explanation

The **TICKET** entity is a weak entity that depends on **BOOKING**. The composite key, **Booking ID** and **Unique Reference Code**, uniquely identifies each ticket within a **BOOKING**. **Booking ID** provides the owning context, and **Unique Reference Code** acts as the discriminator for the individual **TICKET**s within that booking. All **TICKET** attributes depend on the full composite key, and no other component alone is sufficient as a booking may have multiple tickets and the reference code is not globally unique. There are no partial dependencies and no transitive dependencies. Within these stated assumptions, the only functional dependency within **TICKET** has the candidate key as its determinate. Therefore, **TICKET** satisfies BCNF and does not require further decomposition.

10. **SEAT RESERVATION**

Functional Dependencies

Ticket Reference Code -> **Seat Number**, **Boarding Station Code**, **Coach Number**, **Reservation Status**, **Class of Travel**, **Destination Station Code**, **Unique Train Number**

Explanation

SEAT RESERVATION is a strong entity with **Ticket Reference Code** as its primary key. It also serves as a foreign key to **TICKET** when there is a one-to-one relationship, meaning each reservation corresponds to only one **TICKET**. All reservation-specific attributes are fully determined by **Ticket Reference Code**. With a single- attributable key, there are no partial dependencies, and assuming no non-key attribute determines another, there are no transitive dependencies. Under these assumptions, all nontrivial dependencies have a candidate key as the determinant, so **SEAT RESERVATION** is in BCNF.

11. PAYMENT

Functional Dependencies

Payment ID -> Booking ID, **Payment Amount**, **Payment Status**, **Currency**, **Payment Method**, **Reference Code**, **Payment Time and Date**

Explanation

The **PAYMENT** entity is a strong entity identified by **Payment ID**, the primary key, which uniquely distinguishes each transactional record within the system. All payment related descriptors such as **Payment Status**, **Currency** and **Payment Method** are fully and exclusively dependant on **Payment ID**. Due to the key being non-composite, partial dependencies are precluded, furthermore, the database system is designed to avoid non-key determinants, for example **Payment Method** does not determine **Payment Status**. This therefore eliminates transitive dependencies among non-key attributes. Although **PAYMENT** participates in referential associations with **BOOKING** by **Booking ID**, this link does not alter the candidate key and determinant for all attributes stored in the relation. As a result, every nontrivial functional dependency has a super key and therefore the **PAYMENT** entity satisfies BCNF without the need for decomposition.

Normalisation across all the entities

Across all these entities, normalisation ensures that each relation captures a single, well-defined these with attributes fully dependant on appropriate keys, minimising redundancy and update anomalies. **PASSENGER** and **BOOKING** use single attribute primary keys, so all their descriptive attributes depend solely on these keys, with no non-key determinants so therefore satisfying BCNF. **TICKET**, a weak entity, relies on the composite key **Booking ID** and **Unique Reference Code** to uniquely identify each **TICKET** with these attributes, depending on the full composite and therefore satisfying BCNF. **SEAT RESERVATION** is another strong entity with the functional dependency stemming from **Ticket Reference Code** again satisfying BCNF as no non-key determinants are introduced. **PAYMENT** uses **Payment ID** as the primary key, which again determines the other entities and therefore satisfices BCNF.

Deliverable 4- Analysis of the schema in terms of performance, and representation power

The railway reservation schema is normalised so that each relation captures a single concept with attributes fully dependent on appropriate keys, supporting integrity and clarity. **PASSENGER** and **BOOKING** use single-attribute keys (**Email**, **Booking ID**), while **TICKET** relies on the composite key (**Booking ID**, **Unique Reference Code**). **SEAT RESERVATION** uses **Unique Reference Code** as a primary key that is also a foreign key to **TICKET** in a one-to-one relationship, and **PAYMENT** is keyed by **Payment ID**. These choices eliminate partial and transitive dependencies and keep determinants as candidate keys, satisfying BCNF under the stated assumptions. Domain dependencies (for example **Station Code** -> **Station Name**) remain in authoritative tables to avoid non-key determinants that would break BCNF, reducing redundancy and update anomalies and enabling reliable joins across booking retrieval, seat allocation, and timetable queries.

Real-world bottlenecks will centre on high-demand paths such as seat allocation, segment-based availability checks, and schedule lookups by date and time under heavy concurrency. Contention arises when many users target the same services and seats leading joins across schedule stop sequences can to use extensive CPU and therefore increase latency. To mitigate this, all foreign keys should be indexed to add composite indexes to aligned filters and joins. Beyond indexing, reorganisations can improve performance without compromising normalisation. Vertical partitioning large or rarely accessed attributes should be done, for example payment data, to keep companion tables major rows small. Horizontally partitioning should be done to high-volume tables such as **BOOKING**, **TICKET**, **SEAT RESERVATION**, **PAYMENT**) by **Journey Date** or **Booking ID** to localise scans, enable partition pruning and parallelism, and simplify maintenance. Precomputation of expensive aggregates should also be done, such as seat availability per service. Any surrogate keys should be kept for narrow joins to ensure all referential paths are indexed.

Production performance within this railway system will depend not only on the well normalised schema but also on effective concurrency control, workload management, and strong security practices. For example, optimistic or pessimistic locking alongside unique seat constraints could be used to prevent double bookings. Traffic surges should be prevented by introducing queues to the database system during busy periods and personal information should be encrypted. Taken together, these operational safeguards will compliment normalisation and deliver realistic, resilient performance and compliance grade security within a live environment.

As the reservation system evolves, additional tables will be required to address other aspects, such as inventory, pricing and notifications. For example, a **SERVICE CAPACITY** table would enable strong availability management such as promotions support and rule driven pricing. Furthermore, operational resilience could be improved with disruption and exception timetable structures to represent real world service changes and notification schemas, this would then also standardise customer communications. Collectively, these extensions to the current schema would decouple concerns and improve contingency plans for complex scenarios. This would therefore preserve scalability and maintainability, allowing the core normalised schema to grow into a production-grade platform without extensive rework.

Deliverable 5- SQL queries using the schema developed after normalisation

1. Seats booked in second class between Loughborough and London on a given date.

```
SELECT
  ts.UniqueTrainNumber,
  COUNT(DISTINCT sr.TicketReferenceCode) AS SecondClassSeatsBooked
FROM TrainSchedule ts
JOIN IntermediateStation is_l ON is_l.UniqueTrainNumber = ts.UniqueTrainNumber
JOIN IntermediateStation is_lo ON is_lo.UniqueTrainNumber = ts.UniqueTrainNumber
JOIN SeatReservation sr ON sr.UniqueTrainNumber = ts.UniqueTrainNumber
JOIN Ticket t ON t.UniqueReferenceCode = sr.TicketReferenceCode
WHERE ts.JourneyDate = DATE '2025-12-01'
  AND is_l.StationCode = (SELECT StationCode FROM Station WHERE StationName =
'Loughborough')
  AND is_lo.StationCode = (SELECT StationCode FROM Station WHERE StationName =
'London')
  AND is_l.SequenceNumber < is_lo.SequenceNumber
  AND t.ClassOfTravel = 'Second'
GROUP BY ts.UniqueTrainNumber;
```

2. Find all trains between Loughborough and London which depart Loughborough between 18:00 and 21:00.

```
SELECT DISTINCT ts.UniqueTrainNumber
FROM TrainSchedule ts
JOIN IntermediateStation is_l ON is_l.UniqueTrainNumber = ts.UniqueTrainNumber
JOIN IntermediateStation is_lo ON is_lo.UniqueTrainNumber = ts.UniqueTrainNumber
WHERE is_l.StationCode = (SELECT StationCode FROM Station WHERE StationName =
'Loughborough')
  AND is_lo.StationCode = (SELECT StationCode FROM Station WHERE StationName =
'London')
  AND is_l.SequenceNumber < is_lo.SequenceNumber
  AND is_l.DepartureTimeAtSource BETWEEN TIME '18:00' AND TIME '21:00';
```

3. Find all trains between Loughborough and London which depart Loughborough between 18:00 and 21:00 but only include trains that have at least one seat available in First-class

```
SELECT DISTINCT ts.UniqueTrainNumber
FROM TrainSchedule ts
JOIN IntermediateStation is_l ON is_l.UniqueTrainNumber = ts.UniqueTrainNumber
JOIN IntermediateStation is_lo ON is_lo.UniqueTrainNumber = ts.UniqueTrainNumber
WHERE is_l.StationCode = (SELECT StationCode FROM Station WHERE StationName =
'Loughborough')
  AND is_lo.StationCode = (SELECT StationCode FROM Station WHERE StationName =
'London')
  AND is_l.SequenceNumber < is_lo.SequenceNumber
  AND is_l.DepartureTimeAtSource BETWEEN TIME '18:00' AND TIME '21:00'
  AND EXISTS (
    SELECT 1
    FROM SeatReservation sr
    JOIN Coach c ON sr.CoachNumber = c.CoachNumber
    WHERE sr.UniqueTrainNumber = ts.UniqueTrainNumber
      AND c.ClassType = 'First'
  );
```

4. Write an SQL statement to determine the number of seats booked in each train for each year. This should return Train Number, year, number of ticket in second class and number of tickets in first class. Return 0 zero for any cases that trains or services have no bookings

```
SELECT
  ts.UniqueTrainNumber,
  EXTRACT(YEAR FROM ts.JourneyDate) AS Year,
  COUNT(CASE WHEN t.ClassOfTravel = 'Second' THEN 1 ELSE NULL END) AS
SecondClassTickets,
  COUNT(CASE WHEN t.ClassOfTravel = 'First' THEN 1 ELSE NULL END) AS
FirstClassTickets
FROM TrainSchedule ts
LEFT JOIN SeatReservation sr ON sr.UniqueTrainNumber = ts.UniqueTrainNumber
LEFT JOIN Ticket t ON t.UniqueReferenceCode = sr.TicketReferenceCode
GROUP BY ts.UniqueTrainNumber, EXTRACT(YEAR FROM ts.JourneyDate)
ORDER BY Year DESC, ts.UniqueTrainNumber;
```

5. Write an SQL query to determine the Train Number which has generated the highest total amount of sales across all years. This should return the train number and its source and destination.

```
WITH TrainRevenue AS (
  SELECT
    ts.UniqueTrainNumber,
    SUM(t.TicketCost) AS TotalSales
  FROM TrainSchedule ts
  JOIN SeatReservation sr ON sr.UniqueTrainNumber = ts.UniqueTrainNumber
  JOIN Ticket t ON t.UniqueReferenceCode = sr.TicketReferenceCode
  GROUP BY ts.UniqueTrainNumber
)
SELECT
  tr.UniqueTrainNumber,
  tr.TotalSales,
  ts.SourceStation,
  ts.DestinationStation
FROM TrainRevenue tr
JOIN TrainSchedule ts ON ts.UniqueTrainNumber = tr.UniqueTrainNumber
ORDER BY tr.TotalSales DESC
FETCH FIRST 1 ROW ONLY;
```

6. For all trains which run between Sheffield and Loughborough, determine the number of tickets booked for each service time the year 2025. This should return the train number, service time and number of tickets. Service time refers to departure time at source station.

```
SELECT
  ts.UniqueTrainNumber,
  is_sh.DepartureTimeAtSource AS ServiceTime,
  COUNT(t.UniqueReferenceCode) AS TicketsBooked
FROM TrainSchedule ts
JOIN IntermediateStation is_sh ON is_sh.UniqueTrainNumber = ts.UniqueTrainNumber
JOIN IntermediateStation is_lb ON is_lb.UniqueTrainNumber = ts.UniqueTrainNumber
JOIN SeatReservation sr ON sr.UniqueTrainNumber = ts.UniqueTrainNumber
JOIN Ticket t ON t.UniqueReferenceCode = sr.TicketReferenceCode
WHERE EXTRACT(YEAR FROM ts.JourneyDate) = 2025
  AND is_sh.StationCode = (SELECT StationCode FROM Station WHERE StationName =
'Sheffield')
  AND is_lb.StationCode = (SELECT StationCode FROM Station WHERE StationName =
'Loughborough')
  AND is_sh.SequenceNumber < is_lb.SequenceNumber
GROUP BY ts.UniqueTrainNumber, is_sh.DepartureTimeAtSource
ORDER BY ServiceTime ASC, ts.UniqueTrainNumber;
```

Conclusion and Future Enhancements

The railway reservation system developed in this project exhibits a disciplined and coherent application of relational database principles. Through precise modelling of trains, schedules, stations, passengers, bookings, tickets, and seat reservations, the design achieves strong structural integrity and clear operational semantics. The comprehensive normalisation of all entities to BCNF mitigates redundancy and anomalies, therefore promoting consistency, scalability, and maintainability as data volumes expand. Enforcement of constraints and foreign keys, and the use of composite keys, and carefully chosen indexing strategies further enhance data fidelity and query efficiency. The SQL queries presented in Deliverable 5 further demonstrate the schema's robustness by enabling diverse analytical and operational capabilities, including route-aware filtering, class-specific capacity measurement and revenue consolidation. Taken together, these components demonstrate that the system is both theoretically rigorous and pragmatically suited to real-world railway operations.

Looking ahead, several targeted enhancements could broaden the system's functional horizon and advance its evolution into a sophisticated, customer-oriented platform. The adoption of dynamic pricing mechanisms, responsive to demand elasticity, seasonal variation, and booking trajectories would strengthen revenue optimisation and improve load distribution. Customer engagement could be deepened through loyalty schemes, reward accrual, and individualised travel recommendations. Seamless integration with mobile applications would enable unified multi-channel booking, real-time service information, and improved accessibility. From an operational standpoint, advanced analytics and machine learning could underpin demand forecasting, route profitability assessment, and predictive maintenance planning. Features such as accessibility profiling, seat-type personalisation, and automated disruption alerts would further refine the passenger experience. Collectively, these enhancements would transition the system from a dependable reservation infrastructure to a strategic digital platform capable of supporting sustained organisational growth, operational resilience, and enhanced customer satisfaction.

Bibliography

Bernstein and Goodman (1980). 'What does Boyce-Codd Normal Form do?', *VLDB*, pp 245-259). Available at: https://www.microsoft.com/en-us/research/wp-content/uploads/2020/12/WhatDoesBCNFdo-VLDB1980.pdf

Jajodia and Springsteel (1983). 'Entity-relationship diagrams which are in BCNF', *International Journal of Computer & Information Sciences*, 12(4), pp. 269- 283. Available at: https://link.springer.com/article/10.1007/BF00991622

Jiang and Naumann (2019). 'Holistic primary key and foreign key detection', *Journal of Intelligent Information Systems*, 54(3), pp. 439- 461. Available at: https://link.springer.com/article/10.1007/s10844-019-00562-z

Noor (2021). 'Functional dependency', *University of Potsdam, Department of Computer Science*. Available at: https://www.researchgate.net/profile/Alaa-Noor/publication/364309560_Functional_Dependency/links/634565469cb4fe44f31d865e/Functional-Dependency.pdf

Vianu (1987). 'Dynamic functional dependencies and database aging', *Journal of the ACM (JACM)*, 34(1), pp. 28-59. Available at: https://dl.acm.org/doi/abs/10.1145/7531.7918